SimDatabase_Documentation

Document Version: 1.1

Change History

Version	Changes
1.1	Added **popcornfxBloodFootprintName** as an optional property when specifying popcorn effects as a tag property. Fixed naming of section 3.4.2.1.

1 Introduction

The purpose of this document is to identify content that may be useful for modders related to simdata.xml. The Simulation Database's purpose is to represent the sync safe version of data that can impact replays and multiplayer behavior.

Each element in the simdata.xml is either a <visual> or an <animxml> type tag.

2 Visuals

Any model that can become the "main visual" used by an AnimXML should appear as a visual entry. This is a combination of the information to define bounds and collisions for the data with

position changes that would be applied when certain animations play.

2.1 <visual> Node Attributes

The pair of the file and type attributes is used as a unique key.

Attribute	Description
file	Relative path to the file this visual is referencing.
type	Type accepts these items as valid options: • composite • model • popcorn

2.2 <visual> Child Nodes

The following are nodes that can appear as children to the <visual> tag.

Tag	Description
attachpoints	Contains a list of any attach points as child <attachitem> tags. See 2.3 for the definition.
impactpoints	Contains a list of any impact points as child <impactpoint> tags. See 2.4 for the definition.
hasdestruction	Boolean tag that should have <hasdestruction>true</hasdestruction> when there is supported destruction.
boundsmin	X,Y,Z as 3 floats that define the minimum bounds. This is used for some object collisions and to control how the heightfield is represented in volume. These are tight bounds around the object.
boundsmax	X,Y,Z as 3 floats that define the maximum bounds. This is used for some object collisions and to control how the heightfield is represented in volume. These are tight bounds around the object.
heightfield	Expects a group of exactly 32 child nodes, where each node acts like:

	<row>...</row> where it contains a list of heights at relative height values. The list expects 32 integer numbers, which can have values between 0 and 31. The result is a height map that forms a collision mesh for projectiles.
heightfield64	Expects a node <a>... and a node ... where the data is split exactly in half. The data is ordered the same way as the heightfield, with 32 * 32 total values and values between 0 and 31. This is stored as an integer array that was cast to char, then converted to base64.
composite	Deprecated. Sets the type to composite and the file path to the value of the node. Use the attributes on the visual node instead.
model	Deprecated. Sets the type to model and the file path to the value of the node. Use the attributes on the visual node instead.
popcorn	Deprecated. Sets the type to popcorn and the file path to the value of the node. Use the attributes on the visual node instead.

2.3 <attachitem> Child Nodes

The following are nodes that can appear as children to the <attachitem> tag.

Tag	Description
name	Name for the attach point. Requires a value that is not whitespace and not empty.

mat	Contains four child nodes to represent the position in space relative to the unit's origin:
	`<mat>`
	`  <R>1.0000,0.0000,0.0000</R>`
	`  <U>0.0000,1.0000,0.0000</U>`
	`  <F>0.0000,0.0000,1.0000</F>`
	`  <T>0.0000,0.0000,0.0000</T>`
	`</mat>`
	R = Right
	U = Up

	F = Forward T = Translation
type	Type is optional. If it is not specified, the type will default to Static (e.g., that the attach point will remain in the fixed mat location relative to the unit's origin) You can set this to "BoneAttached" to flag that some bone animates it.
DummyType	Dummy bone types are used as helpers for attaching to things used by general in-game properties. This is partially inferred from the name, but it should be specified here if there is a type it should match. The following list shows the valid dummy types: • GenericAttachPoint • Forehead • LeftHand • RightHand • Spine • FrontChest • LeftFoot • RightFoot • PickupAndThrow • HitPointBar • GarrisonFlag • GatherPoint • Fire • Corpse • LaunchPoint • ConstructionHammer • ConstructionLifting • ConstructionSaw • ConstructionStaking • BoatWake • FarmCorn

	• FarmFeed • Relic • vfx_top • vfx_pestilence • ChargeAttackLaunchPoint • AttackRelic
tracks	Contains a list of any animation tracks as child <track> tags. See 2.3.1 for the definition.

2.3.1 <track> Child Nodes

The following are nodes that can appear as children to the <tracks> tag. Note that the values used here are pre-compressed bone hierarchies so that the playback is a simple interpolated position, move, or rotation. Scale has no impact on simulation behavior, so it is not stored here.

Tag	Description
anim	Relative path to the animation file that this track represents. When that animation is playing, and this attach point is queried by the simulation, it will look up the data defined by the track as an override to the default attach point mat.
frames	Expected count of frames. Only used for validation of the fields below. Note that this is not applied as a restriction, and interpolation on each translation/rotation set of frames is interpolated relative to the proportional position in the animation.
t	Comma-separated list of floats that represent the change to translation at proportional points in time between the start and end of the animation. The game does not use this for compression reasons, but modders can use it if desirable. If the translation is unchanged over the duration, it can be a single value, for example. Or it can be left out entirely if the value is unchanged from the default mat. Requires a list of floats that is divisible by 3 to read in groups of “x,y,z”.
t64	Groups of 4 floats (because Vector has a W component) that have been cast from an array of floats to an array of chars,

	then compressed into base64 for safe storage in JSON. Behavior is the same as “t” except it is compressed raw data.
r	Same behavior as “t” except that it represents rotation. Stored as a list of “x,y,z,w” as a Quaternion value.
r64	Same as “t64” except this is storing rotation similar to “r”.

2.4 <impactpoint> Child Nodes

The following are nodes that can appear as children to the <impactpoints> tag. Impact points are used to define where projectiles are fired toward.

Tag	Description
position	Position relative to the unit’s origin in local model space. X,Y,Z as three float values.
flags	No functional purpose. Defaults to “all” if not specified or lookup fails. Valid strings include: • all • low • high • titan
impacttype	Impact Type can change the type of sound that may play when the impact point is hit and the kind of VFX that may show. Valid options include: • Unarmoured • Armoured • Metal • Wood • Stone • Tree • TerrainGrass • TerrainWater

	• TerrainDirt • TerrainStone • TerrainSnow • TerrainSand • ExitBuilding • Titan

3 AnimXMLs

AnimXML nodes represent the information needed to work out what visual is currently displayed by a model and any events that play during animations. They are sort of like a compressed version of the complete AnimXML data.

3.1 <animxml> Node Attributes

Attribute	Description
file	Unique file name reference matching the AnimXML file's relative path.

3.2 <animxml> Child Nodes

The following are nodes that can appear as children to the <animxml> tag.

Tag	Description
file	Deprecated way to set the file name. Use the attribute instead.
visuallogics	Defines the logic used to determine which visual is displayed. See section 3.3 for more details. Animations reference these.
animations	Defines the animations that can play as part of the AnimXML, which includes dictating which set of visual logics to use. See section 3.4 for more details.
alsouseslogic	Defines logic types used by the AnimXML data file that have not been included for any reason as part of the compressed visual/animationlogic. This is done to ensure that when specific simulation side events occur, such as

	age changes, visuals can be refreshed, depending on the logic types used. This is done by just doing the following, where each additional logic type is just listed as <logictype/> inside the parent tag. `<alsouseslogic>   <Culture/> </alsouseslogic>`
animlogics	Defines the logic used to compute animations. See section 3.5 for more details.

3.3 <visuallogics> Child Nodes

The <visuallogics> node contains one or more <visualmap> nodes. In almost every situation, there will only be one visual map. Villagers require a second visual map because they have more complex logic that swaps between both visuals and animations for male and female variations. The <visualmap> is referenced as default 0, and to reference the other indexes, the <animinfo> tags use <visualindex>1</visualindex>, where 1 would be the index 1 in the visual map.

Each <visualmap> node has a list of child <visualnode> entries.

Tag	Description
composite	Flags that this is referencing a Composite Model. The relative path to the composite file that should match the path used for a matching <visual> node with the composite. Only listed if this IS a composite. Mutually exclusive with other visual types.
model	Flags that this is referencing a TMModel. The relative path to the model file that should match the path used for a matching <visual> node with the model. Only listed if this IS a model. Mutually exclusive with other visual types.
null	Flags that this is a “show nothing” type state. Mutually exclusive with other visual types.

popcorn	Flags that this is referencing a Popcorn VFX asset. The relative path to the popcorn file that should match the path used for a matching <visual> node with the popcorn. Only listed if this IS a popcorn asset. Mutually exclusive with other visual types.
logic	Consists of a minimal subset of logic required to determine that this visual should show based on logic that appears in the AnimXML file. Each child of the <logic> tag specifies: <logictype>logicvalue</logictype> These should match exactly what is used in the AnimXML for consistency. For details on all the logic types and expected values, you can look at the logic tag information in the AnimXML documentation.

3.4 <animations> Child Nodes

The <animations> node contains one or more <animinfo> nodes. Each <animinfo> node defines a unique animation type that the AnimXML uses. The following tags can be found as children.

Tag	Description
name	Name of the animation. E.g. “Bored”
visualindex	Defaults to 0 if not specified. This specifies which <visualmap> index is used by this animation.
versioncount	Should match the count of child nodes in versions.
versions	The versions node contains a list of <version> nodes. See 3.4.1 for the definition of these nodes.

3.4.1 <version> Child Nodes

The <version> nodes contain the following children.

Tag	Description
file	The relative path to the animation file name.
duration	The time in seconds that the animation plays for.
weight	Defaults to 1 if not specified. Currently, it does nothing as we don't actively use the weight system anywhere.
tags	Contains a collection of tag events that match what AnimXMLs define. This is the real version of what triggers those events because the simulation owns them. See 3.4.2 for more details about the child <tag> nodes.

3.4.2 <tag> Child Nodes

The <tag> nodes contain the following children. Tag nodes are used to spawn VFX, play sounds, etc.

Tag	Description
type	Type expects a valid tag event type. Look in the AnimXML documentation at section 3.3.1 for details about the types of tag events. And the following sections define valid property types for each.
position	Value between 0 and 1 representing the proportion of the animation when the event occurs.
properties	Includes a list of <propertyname>propertyvalue</propertyname> children. Properties are not required if the event type doesn't need them.

3.4.2.1 Tag Types and Valid Properties

Type	Valid Properties
SpecificSoundSet	• checkVisible = boolean • checkOwner = boolean • looping = boolean • allowOverlap = boolean

	• set = string matching the name of a sound set
CameraShake	• force = float • duration = float
DestroyUnitType	• radius = float • unittype = string as a unit type name.
BeginTurretRotation	• duration = float
Particles	• popcornfxName = string relative path reference to popcorn effect • bone = name of bone to attach to. • movewithbone = boolean • stopwithanim = boolean • adjustduration = boolean • popcornfxBloodFootprintName = string relative path reference to popcorn effect
SpawnTempVisual	• composite = string relative reference to composite model • bone = name of bone to attach to. • duration = float
ShadingFade	• end = float • shadingType = Valid shader type name.
Shading	• factor = float • shadingType = Valid shader type name.

3.5 <animlogics> Child Nodes

The <animlogics> node contains one or more <animmap> nodes. Each <animmap> node defines a unique group of animations. The following tags can be found as children in each <animmap>. Animlogics is only necessary to define when the set of animations changes, dependent on logic, E.g., when there are multiple sub-models with their own unique sets of animations, such as male and female villager variations.

Tag	Description

animfirst	Index in the <animations> of the first <animinfo> used by this animation group.
animlast	Index in the <animations> of the last <animinfo> used by this animation group.
logic	Contains a list of <logictype>logicvalue</logictype> tags similar to how the <visuallogics> works. These are used to determine which of the <animlogics> values is used.